University of Basel

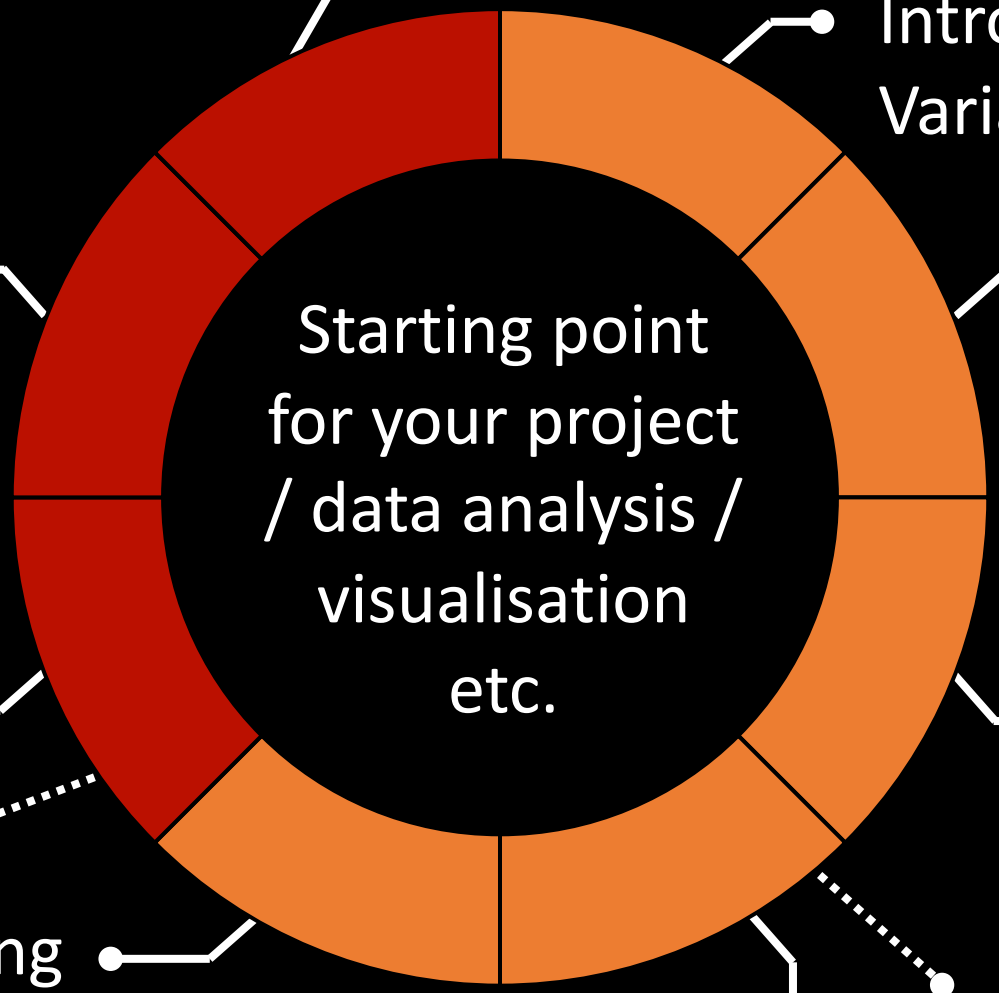
Python Crash Course

Maxim Samarin

18th & 19th February 2021

Please: + for class; + for questions

Overview



Advanced Examples

Introduction,
Variables & Arithmetic

Data Structures

Conditional
Statements

Lunch break
12:30 - 14:15

Iterations

Plotting

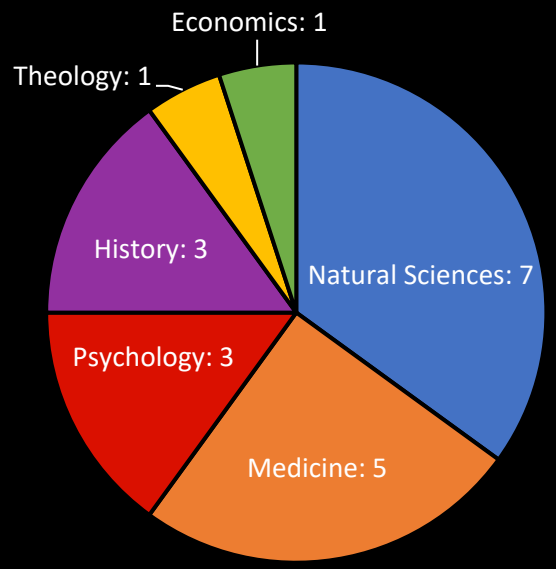
NumPy & Pandas

Subroutines &
Object-oriented
Programming

↑ Friday | Thursday →

Your Experience and Goal

- What is your programming experience so far and an intermediate goal for you?



Muhammad Maladoh ⇢ Catherine ⇢ Frederic ⇢ Alessandro ⇢ Luca ⇢ Laura ⇢ Karolina ⇢ Yunrui ⇢ Nami ⇢ Jana ⇢ Robert ⇢ Samuel ⇢ Judith ⇢ Darja ⇢ Lukas ⇢ Muhannad ⇢ Libin ⇢ Ronja ⇢ Joschka ⇢ Anja

My Programming Career

- School classes programming in Java
- ETH zürich — Studying Physics: C++, MatLab, Python
- CERN — Project: Analysis & Graphical User Interface in C, Python
- ENS DE LYON — Physics modelling in Fortran, Python
- ETH zürich — M.Sc. thesis: Data Mining & simulations in R, Python
- University of Basel — Ph.D. thesis: Machine / Deep Learning research in Python

Python vs. other Languages

- Python is a dynamically typed programming (script) language
- Code is interpreted
- A lot of details *under the hood*
- Zen of Python: Simple, explicit, sparse, readable, practical

→ Fast prototyping, quick to learn

- Statically typed: fix type and object
- Code is compiled
- A lot of explicit control
- Efficient implementations, elaborate syntax

→ Fast and powerful codes

Python & C++ Examples

```python
name = input("Please enter your name: ")
print("Good morning,", name)
```

```cpp
#include <iostream>
#include <string>

using namespace std;

int main() {
    string name;

    cout << "Please enter your name: ";
    cin >> name;
    cout << "Good morning, " << name << endl;

    return 0;
}
```

- Both have the same output:

  Please enter your name: Monty
  Good morning, Monty

Content Python 3 Programming

1. Variables and Arithmetic
2. Data Structures
3. Conditional Statements
4. Iterations
5. Plotting
6. NumPy & Pandas
7. Subroutines and OOP
8. Advanced Project Examples

Start your Working Environment

1. Access the course environment via Noto link provided in E-Mail and sign in with your SWITCH edu-ID ("*Use your Switch AAI login*")
2. Or: Download new material, start Jupyter Lab and open notebooks

- Suggestions on how to work in this course:

  1 + 2 : Follow presentation, while executing scripts yourself, making adjustments and notes in your own notebook

  1 : Follow presentation, switch to programming environment for exercises

Content Python 3 Programming

1. Variables and Arithmetic
2. Data Structures
3. Conditional Statements
4. Iterations
5. Plotting
6. NumPy & Pandas
7. Subroutines and OOP
8. Advanced Project Examples

Content Python 3 Programming

1. Variables and Arithmetic
2. Data Structures
3. Conditional Statements
4. Iterations
5. Plotting
6. NumPy & Pandas
7. Subroutines and OOP
8. Advanced Project Examples

Content Python 3 Programming

1. Variables and Arithmetic
2. Data Structures
3. Conditional Statements
4. Iterations
5. Plotting
6. NumPy & Pandas
7. Subroutines and OOP
8. Advanced Project Examples

Content Python 3 Programming

1. Variables and Arithmetic
2. Data Structures
3. Conditional Statements
4. Iterations
5. Plotting
6. NumPy & Pandas
7. Subroutines and OOP
8. Advanced Project Examples

Content Python 3 Programming

1. Variables and Arithmetic
2. Data Structures
3. Conditional Statements
4. Iterations
5. Plotting
6. NumPy & Pandas
7. Subroutines and OOP
8. Advanced Project Examples

Content Python 3 Programming

1. Variables and Arithmetic
2. Data Structures
3. Conditional Statements
4. Iterations
5. Plotting
6. NumPy & Pandas
7. Subroutines and OOP
8. Advanced Project Examples

Content Python 3 Programming

1. Variables and Arithmetic
2. Data Structures
3. Conditional Statements
4. Iterations
5. Plotting
6. NumPy & Pandas
7. Subroutines and OOP
8. Advanced Project Examples

Different Ways to Execute Scripts

- Jupyter Notebooks (as in this class): Document-style, combining executable scripts with rich documentation and formatting; LaTeX can be used
- Terminal: Execute scripts via the terminal
  1. Write your script in an editor and save it to a *my_script.py* file
  2. Execute the script in the terminal with *python my_script.py*
- Spyder: Integrated development environment (IDE) combining an editor with a terminal and other useful functionalities
- Or any other editor / IDE such as Atom, Sublime Text, PyCharm etc.

Selection of References

- Quick overview with interactive tutorials on some basic topics (similar to the course) and more advanced concepts: https://www.learnpython.org/
- More detailed overview with interactive tutorials on a lot of topics: https://www.w3schools.com/python/default.asp
- One of my favourites with tutorials on specific (advanced) topics, easy-to-read books: https://www.realpython.com/
- Overview of resources for beginners: https://wiki.python.org/moin/BeginnersGuide

Suggestions for the Feedback

- Zoom course appropriate?
- What did / didn’t you like about Noto?
- Content appropriate, anything missing for you?
- Too fast or slow, shallow or deep?
- Insightful exercises and advanced examples?

Thank you and good luck as new Pythonistas! ☺

Some Additional Links

- Download the course material from https://github.com/samarinm/pythonCC
- On script languages: https://en.wikipedia.org/wiki/Scripting_language
- On the difference between interpreter and compiler: https://www.programiz.com/article/difference-compiler-interpreter
- On dynamic and static type checking: https://en.wikipedia.org/wiki/Type_system#Type_checking